LAPORAN ARTIKEL TOOLS KALI LINUX “JOHN THE RIPPER”

DOSEN PENGAMPU

Dr.Eng. I Made Agus Dwi Suarjaya, S.T., M.Eng.

DISUSUN OLEH

I Putu Gunamanta Yuana

(2405551100)

MATA KULIAH

Forensik TI (B-2)

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS UDAYANA

BALI

2025

DAFTAR ISI

DAFTAR ISI ..... **ii**

1. John The Ripper **1**

1.1 Latar Belakang adanya John the Ripper 1

1.2 Metode Peretasan John the Ripper 1

1.2.1 Mode Single Crack 2

1.2.2 Mode Wordlist 2

1.2.3 Mode Incremental 3

2. Aturan Mangling **3**

2.1 Pemanfaatan Set Aturan 3

3. Panduan Penggunaan JtR **4**

3.1 Flag Lainnya 4

3.2 Menginterpretasikan Output dan Melihat Hasil 6

4. Perbedaan John the Ripper dengan Hashcat **7**

4.1 Penggunaan CPU dan GPU 7

4.2 Kelebihan JtR dan Hashcat 7

5. Contoh Penggunaan JtR **8**

5.1 Instalasi John the Ripper 8

5.2 Benchmark Performa Komputer 9

5.3 Persiapan Hash 9

5.4 Menggunakan Mode Single 10

5.5 Menggunakan Mode Wordlist 11

5.6 Menggunakan Mode Incremental 12

1. John The Ripper

John the Ripper (JtR) adalah *software open source* gratis dan *cross platform* yang dirancang untuk melakukan peretasan kata sandi. Tujuan utamanya adalah untuk mendeteksi kata sandi yang lemah. Fitur ini menjadikannya sebuah utilitas penting bagi administrator sistem, *penetration tester*, dan auditor keamanan. Awalnya dikembangkan pada tahun 1996 untuk sistem Unix, fungsi utamanya adalah untuk mengaudit *hash* kata sandi *crypt(3) Unix*. Karena fungsinya yang sering digunakan dan mudah untuk dioperasikan, akhirnya *software* ini diintegrasikan ke dalam Kali Linux.

1.1 Latar Belakang adanya John the Ripper

Sistem operasi tidak menyimpan kata sandi dalam bentuk teks biasa, melainkan mereka menyimpan nilai sebuah *hash* kriptografis, yang merupakan hasil dari fungsi matematika satu arah. Tujuan dari peretas kata sandi adalah menemukan teks biasa asli yang menghasilkan *hash* tertentu. JtR digunakan dalam peretasan *offline*. Dalam serangan *offline*, penyerang pertama-tama mendapatkan salinan basis data hash kata sandi, misalnya, file /etc/shadow Linux atau basis data SAM Windows. Proses peretasan kemudian terjadi pada mesin terpisah yang dikendalikan penyerang. Metode ini tersembunyi dan kuat karena tidak menghasilkan log di sistem target, tidak memicu peringatan keamanan, dan tidak tunduk pada kebijakan penguncian akun. Untuk melawan serangan ini, kita dapat menggunakan *salt* kriptografis, ini merupakan sebuah data acak yang ditambahkan ke kata sandi sebelum di-*hash*. Penambahan *salt* memastikan bahwa kata sandi yang identik menghasilkan *hash* yang berbeda, membuat serangan *rainbow table* yang telah dihitung sebelumnya menjadi tidak efektif dan secara signifikan meningkatkan biaya komputasi peretasan. JtR dirancang untuk menangani *hash* yang diberi *salt* secara otomatis.

1.2 Metode Peretasan John the Ripper

John the Ripper memiliki beberapa mode, yakni mode *Single*, *Wordlist*, dan *Incremental*. Sistem ini dioptimalkan untuk mempermudah audit kata sandi. Mode ini sebeneranya mencerminkan proses investigasi yang logis, dimulai dengan mencoba dengan cepat dengan menebak sandi yang paling mungkin dan secara

bertahap beralih ke metode yang lebih intensif sumber daya. Ini pada dasarnya adalah analisis biaya-manfaat yang memprioritaskan peretasan kata sandi terlemah terlebih dahulu dengan upaya komputasi paling sedikit. Akibatnya, seorang profesional keamanan dapat menyimpulkan kekuatan relatif dari kebijakan kata sandi suatu organisasi hanya dengan mengamati mode mana yang paling banyak memecahkan kata sandi. Jika mode "*single*" menghasilkan tingkat keberhasilan yang tinggi, ini menunjukkan sebuah kelemahan kritis.

1.2.1 Mode Single Crack

Ini adalah mode tercepat JtR dan sering kali dijalankan pertama kali. Mode ini tidak menggunakan daftar kata eksternal. Sebaliknya, dia akan secara cerdas menghasilkan kandidat kata sandi berdasarkan informasi yang sudah ada di dalam *file* kata sandi itu sendiri, seperti nama pengguna, bidang GECOS, dan *path* direktori home. Mode ini menerapkan serangkaian besar aturan *mangling* (menambahkan angka, mengubah huruf besar/kecil), yang membuatnya sangat efektif terhadap pengguna yang membuat kata sandi lemah berdasarkan informasi pribadi. Karena hanya menguji kandidat yang ditargetkan ini terhadap *hash* yang sesuai, mode ini secara signifikan lebih cepat daripada mode *wordlist*.

1.2.2 Mode Wordlist

Mode ini menggunakan *file* yang disediakan pengguna yang disebut "wordlist", atau kamus yang berisi kata sandi potensial, satu per baris. JtR melakukan iterasi melalui daftar ini, melakukan *hash* pada setiap kata, dan membandingkannya dengan *hash* target. Kekuatan sebenarnya dari mode ini terbuka saat dikombinasikan dengan *flag* --rules, yang menerapkan aturan *mangling* pada setiap kata dalam daftar, secara eksponensial meningkatkan jumlah tebakan. Ini dapat mengubah kata sederhana seperti "password" menjadi variasi seperti "P@ssw0rd123!". Keberhasilan mode ini sepenuhnya bergantung pada kualitas *wordlist*. Daftar generik adalah titik awal, tetapi membuat *wordlist* khusus yang disesuaikan dengan target, misalnya dengan mengambil data dari situs web perusahaan menggunakan alat seperti CeWL yang akan secara dramatis meningkatkan efisiensi.

1.2.3 Mode Incremental

Ini adalah mode JtR yang paling komprehensif tetapi paling memakan waktu. Mode ini secara sistematis menghasilkan dan menguji setiap kemungkinan kombinasi karakter untuk set karakter dan panjang yang ditentukan, misalnya, semua karakter ASCII. Tidak seperti serangan brute-force biasa, mode incremental JtR dioptimalkan. Ia menggunakan tabel frekuensi karakter file .chr untuk mencoba teks biasa yang mengandung karakter yang lebih umum terlebih dahulu, sedikit meningkatkan peluangnya untuk menemukan kata sandi lebih awal dalam proses. Waktu yang dibutuhkan untuk serangan ini tumbuh secara eksponensial dengan panjang kata sandi, membuatnya mahal secara komputasi dan sering kali tidak praktis untuk kata sandi yang panjang dan kompleks. Biasanya digunakan setelah mode *single* dan *wordlist* gagal.

2. Aturan Mangling

Aturan *mangling* adalah seperangkat perintah yang didefinisikan dalam *file john.conf* yang menginstruksikan JtR tentang cara memodifikasi kata-kata dari *wordlist* sebelum diuji. Ini memungkinkan wordlist kecil berkualitas tinggi untuk menghasilkan jutaan permutasi, menargetkan pola umum manusia dalam pembuatan kata sandi. Aturan-aturan ini mewakili bentuk pengetahuan yang dikodifikasikan tentang psikologi manusia dalam pembuatan kata sandi. Mereka bukan permutasi acak, melakinkan sebuah algoritma yang dirancang untuk meniru perilaku pengguna yang dapat diprediksi, seperti mengganti huruf dengan angka ('e' -> '3'), mengapitalisasi huruf pertama, atau menambahkan tahun saat ini. Transformasi ini selaras dengan saran umum yang diberikan kepada pengguna untuk membuat kata sandi "kuat", dan juga dengan jalan pintas yang dapat diprediksi yang diambil pengguna untuk memenuhi persyaratan kompleksitas. Karena itu, aturan ini bukan hanya fitur teknis; mereka adalah model tertanam dari kebiasaan komposisi kata sandi.

2.1 Pemanfaatan Set Aturan

JtR dilengkapi dengan seperangkat aturan *default* yang kuat untuk mode *wordlist*, yang diaktifkan dengan *flag --rules*. Aturan-aturan ini dirancang untuk

mencakup berbagai modifikasi kata sandi umum. Untuk serangan yang ditargetkan, para profesional keamanan dapat membuat set aturan mereka sendiri di dalam *john.conf* di bawah bagian bernama ``. Hal ini memungkinkan pembuatan aturan berdasarkan kebijakan kata sandi spesifik suatu organisasi. Misalnya, jika kebijakan mengharuskan karakter khusus di akhir, aturan kustom dapat ditulis untuk menambahkan !@#$%^&* ke setiap kata. Sebagai contoh, *wordlist* sederhana ['perusahaan', 'musimpanas', '2024'] dapat diperluas oleh aturan untuk menguji kandidat seperti Perusahaan2024!, musimpanas24, P3rusahaan!, dan lain-lain, yang sangat meningkatkan peluang keberhasilan tanpa memerlukan wordlist berukuran bergigabyte.

3. Panduan Penggunaan JtR

Struktur perintah yang paling penting di sini adalah `john [options][password-file]`. Jika tidak ada opsi yang ditentukan, JtR akan secara otomatis beralih melalui mode *default*-nya. Salah satu fitur paling penting untuk tugas peretasan yang berjalan lama adalah kemampuannya untuk menyimpan kemajuan secara otomatis ke *file* sesi (default: *john.rec*). Ini memungkinkan pengguna untuk menginterupsi dan melanjutkan proses tanpa kehilangan pekerjaan.

3.1 Flag Lainnya

Tabel berikut merangkum *flag* baris perintah yang paling penting.

Flag	Deskripsi	Contoh Penggunaan
`--wordlist=FILE`	Menentukan file kamus untuk mode wordlist	`john --wordlist=rockyou.txt hashes.txt`
`--rules`	Mengaktifkan aturan mangling untuk mode wordlist	`john --wordlist=list.txt --rules hashes.txt`
`--single`	Mengaktifkan mode single crack	`john --single hashes.txt`
`--incremental`	Mengaktifkan mode incremental (brute-force), opsional dengan set karakter tertentu	`john --incremental=Digits hashes.txt`

`--format=NAME`	Memaksa JtR untuk menggunakan jenis hash tertentu (misalnya, nt untuk NTLM), mengesampingkan deteksi otomatis. Penting untuk hash yang ambigu	`john --format=nt windows_hashes.txt`
`--list=formats`	Menampilkan semua format hash yang didukung oleh versi JtR kita	`john --list=formats`
`--show`	Menampilkan kata sandi yang telah dipecahkan dari potfile untuk file kata sandi yang diberikan	`john --show hashes.txt`
`--stdout`	Mencetak kandidat kata sandi yang dihasilkan ke layar (stdout) alih-alih mengujinya, berguna untuk membuat wordlist baru	`john --wordlist=list.txt --rules --stdout > wordlist_baru.txt`
`--restore`	Melanjutkan sesi peretasan yang terinterupsi	`john --restore`
`--status`	Menampilkan status sesi saat ini	`john --status`
`--fork=N`	Membuat N proses untuk memanfaatkan beberapa inti CPU untuk peretasan, alternatif untuk OpenMP	`john --fork=4 hashes.txt`
`--node=M/N`	Membagi beban kerja untuk peretasan terdistribusi manual di beberapa mesin	`john --node=1/2 hashes.txt`
`--test`	Menjalankan benchmark untuk menguji kecepatan peretasan berbagai	`john --test`

	algoritma hash pada mesin kita	
`--users=[-]LOGIN`	Hanya memuat hash untuk pengguna (LOGIN) yang ditentukan. Tanda - di depan berarti mengecualikan pengguna tersebut	`john --users=admin,root hashes.txt`
`--groups=[-]GID`	Hanya memuat hash untuk grup (GID) yang ditentukan	`john --groups=0 hashes.txt`
`--shells=[-]SHELL`	Hanya memuat pengguna dengan shell yang valid/ditentukan. Berguna untuk mengabaikan akun layanan	`john --shells=-/sbin/nologin hashes.txt`

3.2 Menginterpretasikan Output dan Melihat Hasil

Kata sandi yang berhasil dipecahkan (dalam format *hash:plaintext*) disimpan dalam *file* bernama `john.pot`, disebut "potfile" yang terletak di direktori run JtR. JtR menggunakan *file* ini untuk menghindari peretasan ulang *hash* yang sudah terpecahkan pada proses berikutnya. Potfile tidak dimaksudkan untuk dilihat secara langsung. Cara yang benar untuk melihat hasilnya adalah dengan `flag --show: john --show [password-file]`. Perintah ini merujuk silang potfile dengan file kata sandi asli dan menampilkan kata sandi yang dipecahkan dalam format yang ramah pengguna. Perintah `--show` dapat digabungkan dengan *flag* lain untuk memfilter *output*, misalnya, berdasarkan nama pengguna `--users=root` atau grup `--groups=0,1`.

4. Perbedaan John the Ripper dengan Hashcat

Perbedaan arsitektur antara JtR yang berpusat pada CPU, dan Hashcat yang berpusat pada GPU adalah konsekuensi langsung dari "perlombaan senjata hashing kata sandi." Desain JtR berakar pada era algoritma yang kompleks dan terikat CPU. Kemunculan *Hashcat* sejajar dengan ledakan populernya GPU dan efektivitasnya terhadap *hash* yang lebih sederhana dan dapat diparalelkan. Keberadaan kedua alat ini adalah bukti dari dua strategi utama dalam *hashing* kata sandi modern: membuat *hash* menjadi kompleks secara komputasi yang mendukung kekuatan JtR versus membuatnya cepat secara komputasi dengan mendukung kekuatan Hashcat. Kedua alat ini bukan hanya berbeda, namun mereka adalah produk dari dan solusi untuk sisi yang berlawanan dari konflik kriptografi selama puluhan tahun.

4.1 Penggunaan CPU dan GPU

JtR terutama adalah alat yang dioptimalkan untuk CPU. Ia sangat cepat pada CPU, terutama untuk *hash* yang kompleks secara algoritmik seperti *bcrypt*, di mana kecepatan *clock* mentah dan set instruksi CPU canggih sangat bermanfaat. Dia juga dapat memanfaatkan GPU melalui OpenCL, tetapi ini bukan fokus desain utamanya. Sedangkan *Hashcat* adalah alat yang berpusat pada GPU, dirancang dari awal untuk memanfaatkan kekuatan pemrosesan paralel masif dari sebuah kartu grafis. Dia secara signifikan lebih cepat daripada JtR pada GPU, terutama untuk hash sederhana seperti NTLM dan MD5, di mana ribuan komputasi dapat dijalankan secara paralel.

4.2 Kelebihan JtR dan Hashcat

Kelebihan JtR:

- Hash Lambat: Alat pilihan untuk meretas bcrypt, scrypt, dan hash yang sulit dipecah secara memori lainnya pada CPU
- Fleksibilitas & Mode Unik: Fitur seperti mode "single crack" dan kemampuan untuk membuat format dinamis yang kompleks secara langsung adalah unik untuk JtR

- Lingkungan Hanya CPU: JtR dapat berjalan tanpa GPU atau driver khusus, membuatnya lebih portabel di lingkungan server atau minimal tertentu

Kelebihan Hashcat:

- Hash Cepat & Peretasan Massal: Standar industri untuk peretasan berkecepatan tinggi NTLM, MD5, SHA-1, dll., pada rig multi-GPU.
- Manajemen GPU Tingkat Lanjut: Penyeimbangan beban dinamis yang unggul di beberapa GPU dengan kecepatan campuran
- Fitur Unik: Fungsionalitas "brain" Hashcat mengingat semua kandidat yang pernah dicoba di seluruh sesi untuk menghindari pekerjaan yang berulang, yang sangat berharga untuk upaya peretasan berbasis tim

5. Contoh Penggunaan JtR

Sekarang saya akan mencontohkan cara penggunaan John the Ripper menggunakan *enviroment* tools CLI dari WSL (*Windows Subsystem for Linux*). WSL dapat dikatakan merupakan sebuah emulator yang secara resmi di-*support* oleh Windows sehingga kita tidak perlu membuat sebuah *instance virtual machine* menggunakan VirtualBox perangkat lunak lainnya.

5.1 Instalasi John the Ripper

Pada CLI Linux, kita dapat menggunakan perintah `sudo apt install john` untuk melakukan instalasi JtR pada Kali Linux.

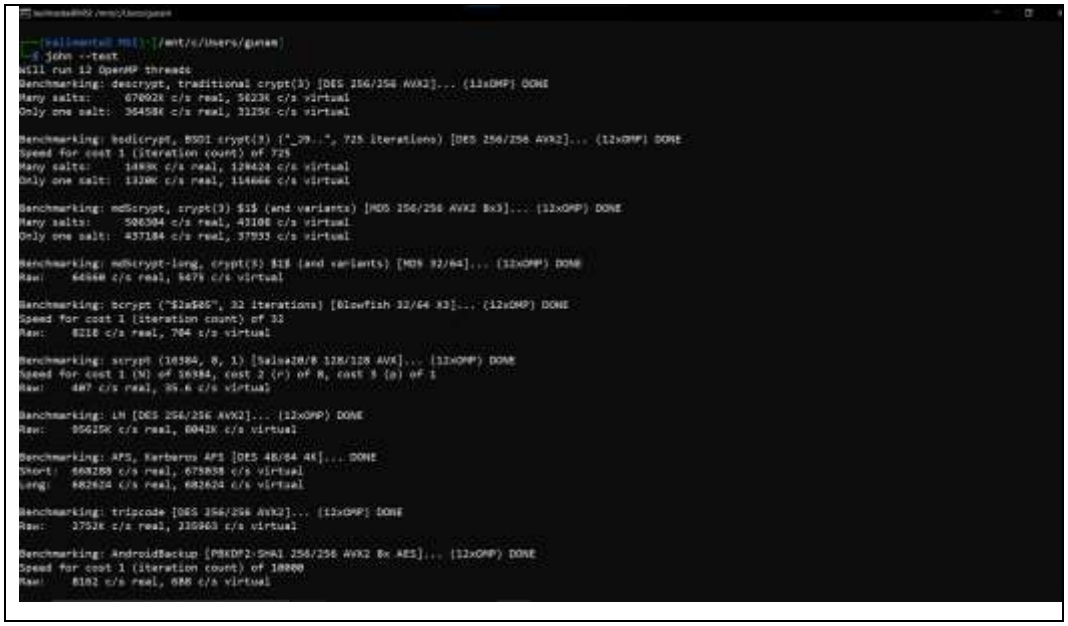
```
┌──(kalimanta㉿MSI)-[/mnt/c/Users/gunam]
└─$ sudo apt install john
[sudo] password for kalimanta:
john is already the newest version (1.9.0-Jumbo-1+git20211102-0kali9).
The following packages were automatically installed and are no longer required:
  libpython3.11-minimal  libpython3.11-stdlib  python3.11  python3.11-minimal
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 184
```

Gambar 5.1 Instalasi John the Ripper

Pada gambar di atas, terlihat bahwa saya telah memiliki *package* JtR sebelumnya.

5.2 Benchmark Performa Komputer

Sebelum memulai serangan, kita dapat mencari tahu seberapa cepat mesin yang kita gunakan untuk dapat memecahkan berbagai jenis *hash* menggunakan perintah `john -test`.



Gambar 5.2 Perintah john --test

Perintah ini akan memberikan gambaran kinerja (*c/s - crypts per second*) untuk setiap algoritma, membantu kita memperkirakan waktu yang dibutuhkan untuk serangan nanti.

5.3 Persiapan Hash

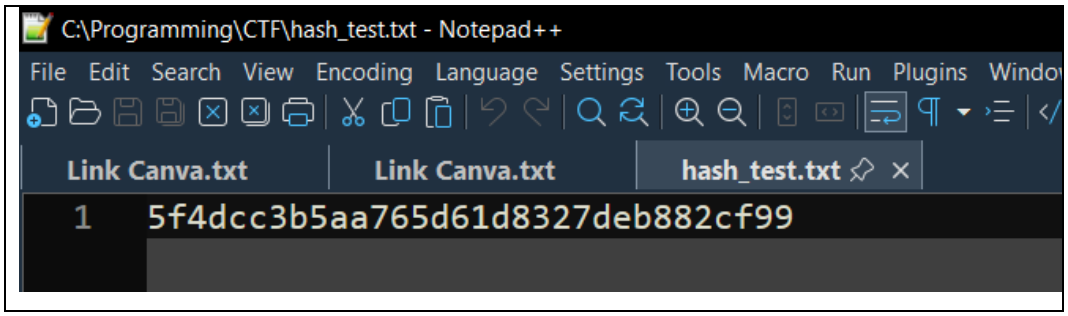
Sebelum kita mulai menggunakan John the Ripper, kita sebelumnya harus memiliki sebuah *file* yang mengandung sebuah *hash* untuk kita *crack*. Pada percobaan ini, saya mempersiapkan sebuah *file* .txt dengan nama “hash_test.txt”.

Gambar 5.3 Direktori programming/ctf/hash_test.txt

File ini nantinya akan kita isi dengan berbagai macam hash, seperti MD5, SHA1, dan bahkan Linux SHA-512 Crypt.

5.4 Menggunakan Mode Single

Pertama-tama saya akan mendemonstrasikan sebuah JtR pada mode *single*.



Gambar 5.4 Hash MD5 "password"

String di atas merupakan sebuah hash MD5 yang ditranslasikan dari kata "password". Sekarang kita akan berupaya untuk memecahkan *string* ini menggunakan mode *single* JtR.

```
┌──(kalimanta㉿ MSI)-[/mnt/c/programming/ctf]
└─$ john --single hash_test.txt --format=raw-md5
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
No password hashes left to crack (see FAQ)

┌──(kalimanta㉿ MSI)-[/mnt/c/programming/ctf]
└─$ john --show hash_test.txt --format=raw-md5
?:password

1 password hash cracked, 0 left
```

Gambar 5.5 Hasil Pemecahan hash_test.txt dengan format MD5

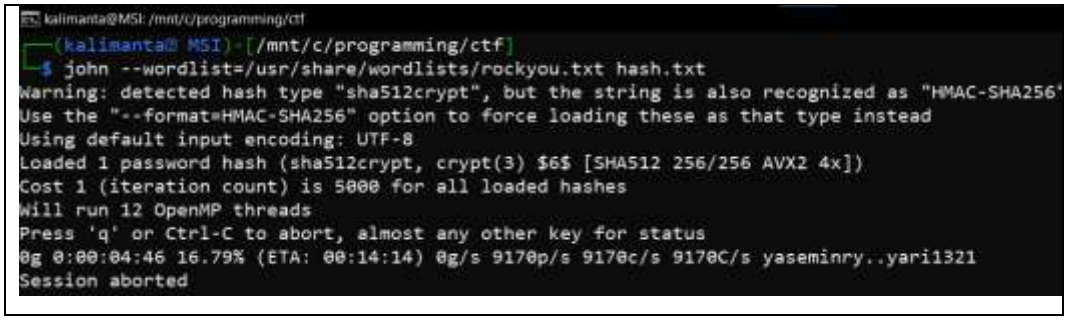
Setelah menjalankan perintah `john --single hash_test.txt --format=raw-md5`, kita mendapati bahwa JtR telah memecahkan dan mendapatkan suatu password, dengan menjalankan `john --show hash_test.txt --format=raw-md5` kita dapat melihat hash tersebut ternyata merupakan sebuah *string* "password".

5.5 Menggunakan Mode Wordlist

Selanjutnya kita akan mendemonstrasikan JtR dalam mode *wordlist*. Dengan mode ini kita dapat menebak sebuah *hash* dengan lebih efisien karena menggunakan sebuah daftar yang dapat menjadi acuan untuk menebak sebuah *hash*.

Gambar 5.6 Hash SHA-512 Crypt

Saya telah mempersiapkan sebuah *hash* SHA-512 Crypt untuk dipecahkan menggunakan JtR. Namun sebelum kita memulai proses pemecahan kita membuat dahulu sebuah daftar kata (*wordlist*) untuk acuan JtR menebak *hash* kita.

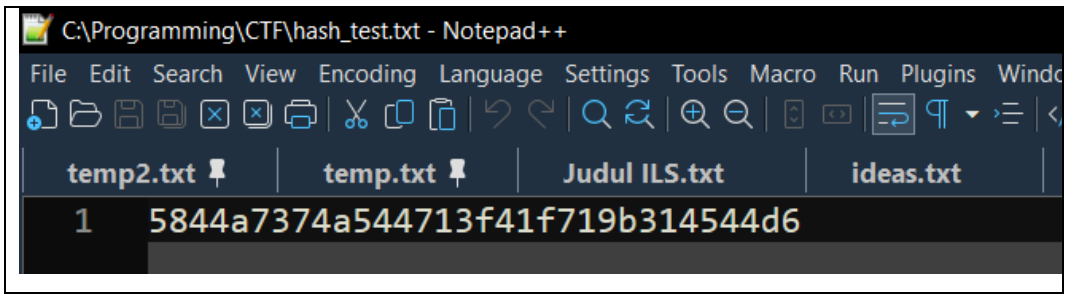


Gambar 5.7 Pemecahan Menggunakan Mode Wordlist

Pada mode ini kita menggunakan kamus dari `rockyou.txt`. Di sini kita seharusnya kita bisa menebak *hash* yang lebih rumit dengan lebih efisien, dan lebih cepat ketimbang mode *incremental*.

5.6 Menggunakan Mode Incremental

Selanjutnya kita akan mendemonstrasikan JtR dalam mode *incremental*. Mode *incremental* menggunakan set karakter (charset) yang telah ditentukan. Kita dapat memberitahu JtR jenis karakter apa yang harus dicoba.



Gambar 5.8 Hash MD5

Sama seperti sebelumnya kita telah menyiapkan sebuah hash MD5 untuk dipecahkan oleh JtR

```
kalimanta@MSI: /mnt/c/programming/ctf

┌──(kalimanta㉿ MSI)-[/mnt/c/programming/ctf]
└─$ john --incremental=Alnum --format=raw-md5 hash_test.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=12
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:06:20  0g/s 64297Kp/s 64297Kc/s 64297KC/s suhtluph..suhtl1cf
Session aborted
```

Gambar 5.9 Pemecahan Hash menggunakan Mode Incremental

Pada tahap ini JtR sedang memecahkan hash kita menggunakan mode *Incremental.* Mode ini akan memakan banyak waktu untuk bisa memecahkan hash ini, meski hanya menggunakan hash MD5, di laptop saya dia masih melakukan pemecahan meski sudah berjalan selama 5 menit.

Gambar 5.10 Bukti Selfie Pengerjaan Tugas